SofInventory

Sistema de Inventarios y Ventas para Ferreterías PYME

Documento de Estándares de Codificación

Módulo Java Web — Usuarios y Proveedores

Control de versiones	
Versión	1.0
Fecha	22 de marzo 2026
Autores	Aprendices ADSO — SENA
Estado	Inicial
Documento elaborado pro por	Alejandro Sepúlveda Daurte
	Lucy Estafany Izquierdo Jaramillo

Contenido

1. Introducción 3
2. Tecnologías utilizadas 3
3. Estructura de carpetas 3
4. Estándares de nomenclatura 4
4.1 Paquetes 4
4.2 Clases 4
4.3 Métodos 4
4.4 Variables y constantes 5
5. Arquitectura MVC 6
6. Estándares de conexión a base de datos 6
6.1 Clase de conexión centralizada 6
6.2 Uso en los DAO — try-with-resources 7
7. Estándares de Servlets 7
7.1 Estructura estándar 7
7.2 Diferencia entre GET y POST 8
8. Estándares de archivos JSP 8
8.1 Etiquetas JSP utilizadas 8
9. Estándares de CSS y HTML 9
9.1 Variables CSS 9
9.2 Sistema de colores 9
9.3 Nomenclatura de clases CSS 10
10. Estándares de comentarios 11
11. Resumen de estándares aplicados 11

1. Introducción

Este documento describe los estándares de codificación aplicados en el desarrollo del módulo Java Web del proyecto SofInventory. El objetivo es garantizar que el código sea legible, mantenible y consistente, de manera que cualquier desarrollador pueda entender y continuar el proyecto sin dificultad.

El módulo fue desarrollado usando Java con Servlets y JSP, conectado a PostgreSQL mediante JDBC, y desplegado en Apache Tomcat 10.1. Se aplicó el patrón de arquitectura MVC (Modelo - Vista - Controlador) para separar claramente las responsabilidades del código.

2. Tecnologías utilizadas

Tecnología	Versión	Uso en el proyecto
Java	24	Lenguaje principal del backend
Jakarta Servlets	EE 10	Controladores de peticiones HTTP
JSP	Jakarta EE	Vistas dinámicas (HTML + Java)
Apache Tomcat	10.1	Servidor web — contenedor de Servlets
PostgreSQL	15+	Base de datos relacional
Driver JDBC	42.7.10	Conexión Java — PostgreSQL
NetBeans IDE	25 / 26	Entorno de desarrollo
HTML5 + CSS3	-	Estructura y estilos del frontend

3. Estructura de carpetas

El proyecto sigue una estructura estándar de aplicación web Java con Ant. Cada carpeta tiene una responsabilidad específica y bien definida:

Carpeta / Archivo	Responsabilidad
src/java/com/sofinventory/utils/	Utilidades — conexión a la base de datos
src/java/com/sofinventory/modelos/	Modelos — clases que representan las tablas
src/java/com/sofinventory/dao/	DAO — consultas SQL (Data Access Object)
src/java/com/sofinventory/servlets/	Servlets — controladores HTTP
web/views/usuarios/	Vistas JSP del módulo de usuarios
web/views/proveedores/	Vistas JSP del módulo de proveedores
web/css/	Hojas de estilo CSS globales
web/WEB-INF/lib/	Librerías externas (.jar)

Carpeta / Archivo	Responsabilidad
web/WEB-INF/web.xml	Configuración del servidor Tomcat

4. Estándares de nomenclatura

4.1 Paquetes

Los nombres de paquetes se escriben siempre en minúscula, usando el dominio invertido como prefijo. No se usan guiones ni mayúsculas.

```
// Correcto
package com.sofinventory.utils;
package com.sofinventory.modelos;
package com.sofinventory.dao;
package com.sofinventory.servlets;

// Incorrecto
package Com.SofInventory.Utils;
package com.sof-inventory.Utils;
```

4.2 Clases

Los nombres de clases usan PascalCase — cada palabra empieza con mayúscula. Los Servlets llevan el sufijo Servlet, los DAO llevan el sufijo DAO, y los modelos usan el nombre de la entidad.

Tipo	Estándar	Ejemplos del proyecto
Modelo	PascalCase	Usuario, Proveedor
DAO	NombreDAO	UsuarioDAO, ProveedorDAO
Servlet	NombreServlet	LoginServlet, UsuarioServlet, ProveedorServlet
Utilidad	PascalCase	Conexion

4.3 Métodos

Los métodos usan camelCase — primera palabra en minúscula, siguientes con mayúscula inicial. Los métodos deben describir claramente la acción que realizan.

Método	Acción	Archivo
listar()	SELECT todos los registros	UsuarioDAO, ProveedorDAO
buscarPorId(id)	SELECT por ID	UsuarioDAO, ProveedorDAO

Método	Acción	Archivo
insertar(obj)	INSERT nuevo registro	UsuarioDAO, ProveedorDAO
actualizar(obj)	UPDATE registro existente	UsuarioDAO, ProveedorDAO
eliminar(id)	DELETE por ID	UsuarioDAO, ProveedorDAO
login(user, pass)	Validar credenciales	UsuarioDAO
getConexion()	Abrir conexión a BD	Conexion
doGet(req, res)	Petición HTTP GET	Todos los Servlets
doPost(req, res)	Petición HTTP POST	Todos los Servlets

4.4 Variables y constantes

Las variables de instancia y locales usan camelCase. Las constantes usan UPPER_CASE con guiones bajos entre palabras.

```
// Variables de instancia (camelCase)
private int    id;
private String nombreCompleto;
private String tipoDocumentoNombre;
private int    creadoPorId;

// Constantes (UPPER_CASE)
private static final String URL      =
"jdbc:postgresql://localhost:5433/db_sofinventory_java";
private static final String USUARIO  = "postgres";
private static final String PASSWORD = "admin123";

// Variables locales (camelCase)
String nombreCompleto = req.getParameter("nombreCompleto");
int    rolId = Integer.parseInt(req.getParameter("rolId"));
```

5. Arquitectura MVC

El proyecto aplica el patrón Modelo — Vista — Controlador (MVC). Cada capa tiene una responsabilidad específica y no debe mezclar lógica de otra capa.

Capa	Archivos	Responsabilidad
Modelo	Usuario.java, Proveedor.java	Representar la estructura de datos de cada tabla
DAO	UsuarioDAO.java, ProveedorDAO.java	Contener todas las consultas SQL (JDBC)
Controlador	UsuarioServlet.java, ProveedorServlet.java, LoginServlet.java	Recibir peticiones HTTP y coordinar Modelo y Vista
Vista	usuarios/index.jsp, proveedores/index.jsp, login.jsp	Mostrar datos al usuario — HTML con código Java

Flujo de una petición HTTP:

```
1. Navegador envía petición HTTP (GET o POST)
2. Tomcat enruta al Servlet según @WebServlet("/ruta")
3. Servlet llama al DAO para consultar o modificar la BD
4. DAO ejecuta SQL con JDBC y retorna objetos del Modelo
5. Servlet pasa los datos al JSP con request.setAttribute()
6. JSP genera el HTML con los datos y lo envía al navegador
```

6. Estándares de conexión a base de datos

La conexión a PostgreSQL se centraliza en la clase Conexion.java. Ningún otro archivo debe tener datos de conexión hardcodeados. Se usa el patrón try-with-resources para garantizar el cierre automático de la conexión y evitar fugas de memoria.

6.1 Clase de conexión centralizada

```
public class Conexion {
    // Constantes en UPPER_CASE
    private static final String URL      =
"jdbc:postgresql://localhost:5433/db_sofinventory_java";
    private static final String USUARIO  = "postgres";
    private static final String PASSWORD = "admin123";

    // Método estático — se llama sin instanciar la clase
    public static Connection getConexion() throws SQLException {
        try {
```

```
            Class.forName("org.postgresql.Driver"); // Carga el driver JDBC
            return DriverManager.getConnection(URL, USUARIO, PASSWORD);
        } catch (ClassNotFoundException e) {
            throw new SQLException("Driver no encontrado: " + e.getMessage());
        }
    }
}
```

6.2 Uso en los DAO — try-with-resources

```
// Correcto — try-with-resources cierra la conexión automáticamente
try (Connection con = Conexion.getConexion();
     PreparedStatement ps = con.prepareStatement(sql);
     ResultSet rs = ps.executeQuery()) {
    while (rs.next()) {
        lista.add(mapear(rs));
    }
}

// PreparedStatement previene SQL Injection
String sql = "SELECT * FROM usuarios WHERE username = ? AND password = ?";
ps.setString(1, username);
ps.setString(2, password);
```

7. Estándares de Servlets

Los Servlets son los controladores del proyecto. Cada Servlet maneja una entidad del sistema y responde a las peticiones HTTP GET y POST.

7.1 Estructura estándar

```
@WebServlet("/ruta")   // Anotación que define la URL del Servlet
public class NombreServlet extends HttpServlet {

    // DAO se instancia una sola vez
    private final NombreDAO dao = new NombreDAO();

    // GET — al entrar a la página o al hacer clic en Editar/Eliminar
    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
        String accion = req.getParameter("accion");
        req.getRequestDispatcher("/views/nombre/index.jsp").forward(req, res);
    }
```

```
    // POST — al enviar un formulario
    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
        req.setCharacterEncoding("UTF-8"); // siempre al inicio
        res.sendRedirect("ruta?mensaje=creado");
    }
}
```

7.2 Diferencia entre GET y POST

Método	Cuándo se usa	Ejemplo en el proyecto
GET	Listar, buscar, editar, eliminar	/usuarios?accion=editar&id=1
POST	Insertar o actualizar datos	Envío del formulario de usuario

8. Estándares de archivos JSP

Los archivos JSP son las vistas del proyecto. Combinan HTML con código Java para mostrar datos dinámicos de la base de datos. El navegador siempre recibe HTML puro — nunca ve el código Java.

8.1 Etiquetas JSP utilizadas

Etiqueta	Uso	Ejemplo
<% %>	Ejecutar código Java	<% String x = req.getParameter("x"); %>
<%= %>	Imprimir valor en HTML	<%= u.getNombreCompleto() %>
<%@ page %>	Directiva de configuración	<%@ page contentType="text/html;charset=UTF-8" %>
<%@ import%>	Importar clases Java	<%@ page import="java.util.List" %>
<%-- --%>	Comentario (no llega al cliente)	<%-- Este es un comentario JSP --%>

9. Estándares de CSS y HTML

9.1 Variables CSS

Se usan variables CSS (custom properties) para centralizar los colores y valores del sistema de diseño. Esto permite cambiar el tema completo modificando un solo lugar.

```
:root {
    /* Colores principales */
    --primario:    #2563eb;   /* Azul principal — botones, navbar */
    --primario-h:  #1d4ed8;   /* Azul hover */
    --peligro:     #dc2626;   /* Rojo — eliminar, errores */
    --exito:       #16a34a;   /* Verde — mensajes de éxito */

    /* Superficies */
    --fondo:       #f1f5f9;   /* Fondo general de la página */
    --superficie:  #ffffff;   /* Fondo de cards y formularios */
    --borde:       #e2e8f0;   /* Color de bordes */

    /* Tipografía */
    --texto:       #1e293b;   /* Texto principal */
    --texto-suave: #64748b;   /* Texto secundario — labels */

    /* Layout */
    --radio:   8px;           /* Border radius estándar */
}
```

9.2 Sistema de colores

Variable CSS	Valor hex	Uso en el proyecto
--primario	#2563eb	Navbar, botón primario, focus de inputs
--primario-h	#1d4ed8	Hover del botón primario
--peligro	#dc2626	Botón eliminar, alertas de error
--exito	#16a34a	Alertas de éxito, badge activo
--fondo	#f1f5f9	Fondo general de la página
--superficie	#ffffff	Cards, formularios, tabla
--borde	#e2e8f0	Bordes de inputs, tabla, cards
--texto	#1e293b	Texto principal del contenido
--texto-suave	#64748b	Labels, texto secundario

9.3 Nomenclatura de clases CSS

Las clases CSS usan kebab-case — palabras en minúscula separadas por guiones. Se organizan por componente para facilitar la lectura y el mantenimiento.

Clase CSS	Componente	Uso
.navbar	Navegación	Barra de navegación superior
.navbar-marca	Navegación	Nombre del sistema en navbar
.contenedor	Layout	Contenedor principal de la página
.card	Layout	Tarjeta con sombra para secciones
.form-grid	Formulario	Grid responsivo para los campos
.campo	Formulario	Contenedor de label + input
.campo-ancho	Formulario	Campo que ocupa todo el ancho
.btn	Botón	Botón base
.btn-primario	Botón	Botón azul principal
.btn-secundario	Botón	Botón gris para cancelar
.btn-editar	Botón	Botón azul claro — acciones en tabla
.btn-eliminar	Botón	Botón rojo claro — eliminar
.tabla	Tabla	Tabla de datos estilizada
.badge	Badge	Etiqueta de estado redondeada
.badge-activo	Badge	Badge verde — estado activo
.badge-inactivo	Badge	Badge gris — estado inactivo
.alerta-ok	Alerta	Mensaje verde de éxito
.alerta-error	Alerta	Mensaje rojo de error
.login-container	Login	Tarjeta blanca del formulario de login

10. Estándares de comentarios

Los comentarios deben explicar el 'por qué' de una decisión, no el 'qué' hace el código. El código bien escrito se explica solo — los comentarios complementan cuando es necesario.

```
// Separadores de secciones en los DAO
// -- LISTAR TODOS ---------------------------------------------------------

// Comentario de línea — explica algo no obvio
ps.setString(6, u.getPassword()); // Texto plano — proyecto académico

// Comentario de bloque — para lógica compleja
/*
 * El método mapear() convierte cada fila del ResultSet
 * en un objeto Java. Se llama desde listar() y buscarPorId()
 */

// Comentario en JSP — NO llega al navegador
<%-- Este comentario es solo para desarrolladores --%>
```

11. Resumen de estándares aplicados

Elemento	Estándar	Ejemplo
Paquetes	minúscula	com.sofinventory.dao
Clases	PascalCase	UsuarioServlet, ProveedorDAO
Métodos	camelCase	buscarPorId(), listar()
Variables	camelCase	nombreCompleto, tipoDocumentoId
Constantes	UPPER_CASE	URL, USUARIO, PASSWORD
Clases CSS	kebab-case	.btn-primario, .card
Variables CSS	--kebab-case	--primario, --fondo
Archivos JSP	minúscula	index.jsp, login.jsp
Archivos Java	PascalCase	Conexion.java, Usuario.java
Carpetas paquete	minúscula	dao, modelos, utils, servlets

Documento elaborado por los aprendices del programa Análisis y Desarrollo de Software — SENA 2026.